

Minimax and Alpha-Beta Pruning — Tic-Tac-Toe

Run: python P_minimax_alphabeta_tictactoe.py

Program

```
import math

HUMAN = 'O'
AI = 'X'
EMPTY = ' '

def print_board(b):
    for r in range(3):
        print(" " + " | ".join(b[r*3:(r+1)*3]))
        if r < 2:
            print("----+---+---")
    print()

def winner(b):
    lines = [(0,1,2), (3,4,5), (6,7,8),
              (0,3,6), (1,4,7), (2,5,8),
              (0,4,8), (2,4,6)]
    for a,c,d in lines:
        if b[a] != EMPTY and b[a] == b[c] == b[d]:
            return b[a]
    return None

def is_full(b):
    return EMPTY not in b

def minimax(b, depth, is_max):
    w = winner(b)
    if w == AI:
        return 10 - depth
    if w == HUMAN:
        return depth - 10
    if is_full(b):
        return 0

    if is_max:
        best = -math.inf
        for i in range(9):
            if b[i] == EMPTY:
                b[i] = AI
                best = max(best, minimax(b, depth+1, False))
                b[i] = EMPTY
        return best
    else:
        best = math.inf
        for i in range(9):
            if b[i] == EMPTY:
                b[i] = HUMAN
                best = min(best, minimax(b, depth+1, True))
                b[i] = EMPTY
        return best

def minimax_ab(b, depth, is_max, alpha, beta):
    w = winner(b)
    if w == AI:
        return 10 - depth
    if w == HUMAN:
        return depth - 10
    if is_full(b):
        return 0

    if is_max:
        best = -math.inf
        for i in range(9):
            if b[i] == EMPTY:
                b[i] = AI
                best = max(best, minimax_ab(b, depth+1, False, alpha, beta))
                b[i] = EMPTY
                alpha = max(alpha, best)
                if beta <= alpha:
```

```

        break
    return best
else:
    best = math.inf
    for i in range(9):
        if b[i] == EMPTY:
            b[i] = HUMAN
            best = min(best, minimax_ab(b, depth+1, True, alpha, beta))
            b[i] = EMPTY
            beta = min(beta, best)
            if beta <= alpha:
                break
    return best

# Counted versions (for comparing how many nodes each approach visits)
plain_nodes = 0
ab_nodes = 0

def minimax_counted(b, depth, is_max):
    global plain_nodes
    plain_nodes += 1
    w = winner(b)
    if w == AI:
        return 10 - depth
    if w == HUMAN:
        return depth - 10
    if is_full(b):
        return 0
    if is_max:
        best = -math.inf
        for i in range(9):
            if b[i] == EMPTY:
                b[i] = AI
                best = max(best, minimax_counted(b, depth+1, False))
                b[i] = EMPTY
        return best
    else:
        best = math.inf
        for i in range(9):
            if b[i] == EMPTY:
                b[i] = HUMAN
                best = min(best, minimax_counted(b, depth+1, True))
                b[i] = EMPTY
        return best

def minimax_ab_counted(b, depth, is_max, alpha, beta):
    global ab_nodes
    ab_nodes += 1
    w = winner(b)
    if w == AI:
        return 10 - depth
    if w == HUMAN:
        return depth - 10
    if is_full(b):
        return 0
    if is_max:
        best = -math.inf
        for i in range(9):
            if b[i] == EMPTY:
                b[i] = AI
                best = max(best, minimax_ab_counted(b, depth+1, False, alpha, beta))
                b[i] = EMPTY
                alpha = max(alpha, best)
                if beta <= alpha:
                    break
        return best
    else:
        best = math.inf
        for i in range(9):
            if b[i] == EMPTY:
                b[i] = HUMAN
                best = min(best, minimax_ab_counted(b, depth+1, True, alpha, beta))
                b[i] = EMPTY
                beta = min(beta, best)
                if beta <= alpha:

```

```

        break
    return best

def best_move(b, use_ab=True):
    best_val = -math.inf
    move = None
    for i in range(9):
        if b[i] == EMPTY:
            b[i] = AI
            if use_ab:
                val = minimax_ab(b, 0, False, -math.inf, math.inf)
            else:
                val = minimax(b, 0, False)
            b[i] = EMPTY
            if val > best_val:
                best_val = val
                move = i
    return move, best_val

if __name__ == "__main__":
    print("=" * 50)
    print("TIC-TAC-TOE: MINIMAX vs MINIMAX WITH ALPHA-BETA PRUNING")
    print("=" * 50)

    # Board state: AI (X) to move
    board = [
        'X', 'O', 'X',
        'O', 'X', 'O',
        EMPTY, EMPTY, EMPTY
    ]

    print("Current board:")
    print_board(board)

    move1, val1 = best_move(board, use_ab=False)
    print(f"[Plain Minimax] Best move for X: position {move1}, score {val1}")

    move2, val2 = best_move(board, use_ab=True)
    print(f"[Minimax + AlphaBeta] Best move for X: position {move2}, score {val2}")

    board[move2] = AI
    print("\nBoard after AI plays its best move:")
    print_board(board)

    print("-" * 50)
    print("Full game simulation: AI (X) vs AI (O), both playing optimally")
    print("-" * 50)
    board = [EMPTY] * 9
    current = AI
    turn = 1
    while True:
        w = winner(board)
        if w:
            print(f"Result: {w} wins!")
            break
        if is_full(board):
            print("Result: Draw!")
            break
        move, val = best_move(board, use_ab=True) if current == AI else (None, None)
        if current == AI:
            board[move] = AI
        else:
            # HUMAN side also plays optimally, minimizing X's score
            best_val = math.inf
            move = None
            for i in range(9):
                if board[i] == EMPTY:
                    board[i] = HUMAN
                    val = minimax_ab(board, 0, True, -math.inf, math.inf)
                    board[i] = EMPTY
                    if val < best_val:
                        best_val = val
                        move = i
            board[move] = HUMAN
        print(f"Turn {turn}: {current} plays position {move}")

```

```
print_board(board)
current = HUMAN if current == AI else AI
turn += 1

print("-" * 50)
print("Node comparison (from the empty board, X to move):")
board = [EMPTY] * 9
plain_nodes = 0
minimax_counted(board, 0, True)
print(f"Nodes visited without pruning : {plain_nodes}")

board = [EMPTY] * 9
ab_nodes = 0
minimax_ab_counted(board, 0, True, -math.inf, math.inf)
print(f"Nodes visited with alpha-beta : {ab_nodes}")
```

Output

```
=====
TIC-TAC-TOE: MINIMAX vs MINIMAX WITH ALPHA-BETA PRUNING
=====
```

Current board:

```
X | O | X
---+---+---
O | X | O
---+---+---
|   |
```

[Plain Minimax] Best move for X: position 6, score 10
[Minimax + AlphaBeta] Best move for X: position 6, score 10

Board after AI plays its best move:

```
X | O | X
---+---+---
O | X | O
---+---+---
X |   |
```

Full game simulation: AI (X) vs AI (O), both playing optimally

Turn 1: X plays position 0

```
X |   |
---+---+---
|   |
---+---+---
|   |
```

Turn 2: O plays position 4

```
X |   |
---+---+---
| O |
---+---+---
|   |
```

Turn 3: X plays position 1

```
X | X |
---+---+---
| O |
---+---+---
|   |
```

Turn 4: O plays position 2

```
X | X | O
---+---+---
| O |
---+---+---
|   |
```

Turn 5: X plays position 6

```
X | X | O
---+---+---
| O |
---+---+---
X |   |
```

Turn 6: O plays position 3

```
X | X | O
---+---+---
O | O |
---+---+---
X |   |
```

Turn 7: X plays position 5

```
X | X | O
---+---+---
O | O | X
---+---+---
X |   |
```

Turn 8: O plays position 7

```
X | X | O
---+---+---
O | O | X
---+---+---
X | O |
```

Turn 9: X plays position 8

```
X | X | O
---+---+---
O | O | X
---+---+---
X | O | X
```

Result: Draw!

Node comparison (from the empty board, X to move):

Nodes visited without pruning : 549946

Nodes visited with alpha-beta : 20866